

DIARIO DI SVILUPPO

The Painted *Ascent*

Prologo

GIOVANNI MANTEGNA · Unreal Engine 5.8 · 2026

PANORAMICA

Un piccolo angelo in una *grotta buia*

The Painted Ascent è nato dai miei disegni di un piccolo angelo di pietra, metà statua e metà bambino. In tre mesi, da solo, ho trasformato quei disegni in un prologo giocabile, imparando gran parte delle cose lungo la strada.



La prima stanza: un fascio di luce, un puzzle di cristalli e una porta che ancora non si apre.

Il prologo si svolge in una grotta di cristalli luminosi. Passo dopo passo l'angelo impara qualcosa di nuovo: interagire con il mondo, fare il doppio salto, scattare in aria, tirare un pugno, farsi lanciare verso l'alto, colpire con tutta la sua forza. Gli hint restano brevi, perché ogni ostacolo è costruito per avere una sola risposta, ovvia.

In fondo alla grotta, nel buio, lo aspettano un pennello e una tavolozza. L'angelo li prende, le pareti diventano viola e comincia il primo vero combattimento.



L'arena: il Colosso e i suoi Caster aspettano che l'angelo faccia la prima mossa.

Perché l'ho fatto

Vengo dall'ingegneria e dall'intelligenza artificiale, e voglio lavorare nei videogiochi. Questo progetto era il mio modo di scoprire se riuscivo a portare un'idea da uno schizzo fino a qualcosa che le persone possono davvero giocare: progettare il livello, costruire i personaggi, scrivere ogni sistema in Blueprint e far sì che tutto sembrasse un piccolo mondo unico.

3 mesi GIUGNO – SETTEMBRE 2026	**Da solo** DESIGN, GRAFICA, CODICE, UI	**Unreal Engine 5.8** TUTTO IN BLUEPRINT
3 stanze 8 SEZIONI	**2 nemici** COLOSSO E CASTER	**1 cinematica** REALIZZATA IN SEQUENCER

In questo diario

1 Game design e level design

2 Pipeline grafica

3 Animazione

4 Programmazione del gameplay

5 Effetti, luce e interfaccia

6 L'IA nel mio lavoro

7 Guardando indietro

Il prologo è completo e giocabile dall'inizio alla fine. Suoni e musica arriveranno più avanti.
Guarda il gameplay: **youtube.com/watch?v=6CvP8dkcaxc**

Un passo *alla volta*

Tre stanze, divise in otto brevi sezioni. Ogni sezione introduce un'abilità, e l'ostacolo che hai davanti ha una sola risposta, ovvia.

Una risposta

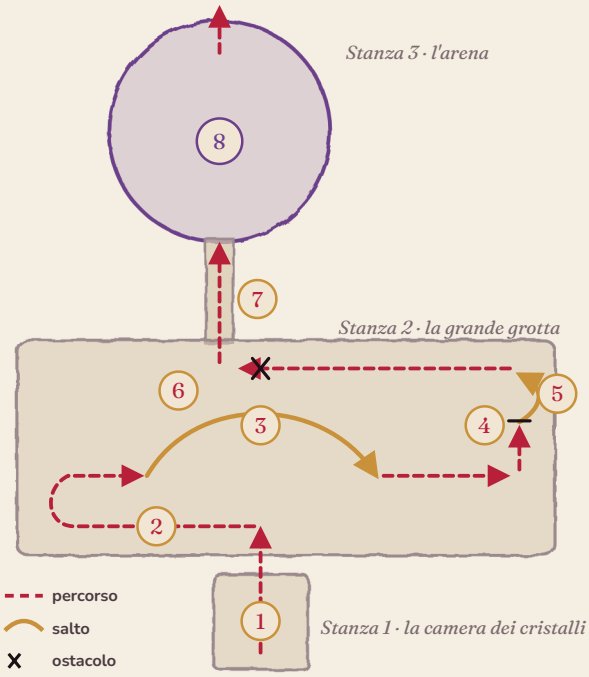
Ogni ostacolo si supera in un solo modo chiaro.

Una cosa alla volta

Un'abilità si impara da sola prima di combinarla con altre.

Forma, non cartelli

La geometria obbliga a usare la meccanica, invece di permetterla soltanto.



1. **Puzzle dei cristalli.** Ruota i cristalli per collegarli con raggi di luce: quando la catena è completa, la porta si apre.
2. **Rocce rotolanti.** Rocce incandescenti ti rotolano incontro: saltale.
3. **Il burrone.** Un potenziamento del salto e il dash ti portano dall'altra parte.
4. **La pietra rossa.** Un pugno la spezza.
5. **Il lancio.** Aspetta dove c'era la pietra: il pavimento ti lancia sulla sporgenza.
6. **Pugno caricato.** Raccogli l'inchiostro, riempi il sigillo, spezza l'ultima roccia.
7. **Il corridoio.** Niente da imparare, di proposito: un respiro prima dello scontro.
8. **L'arena.** Tutto insieme.



Sezione 2: rocce incandescenti sul sentiero.



Sezione 4: la pietra rossa.



Gocce luminose lungo una cengia stretta.

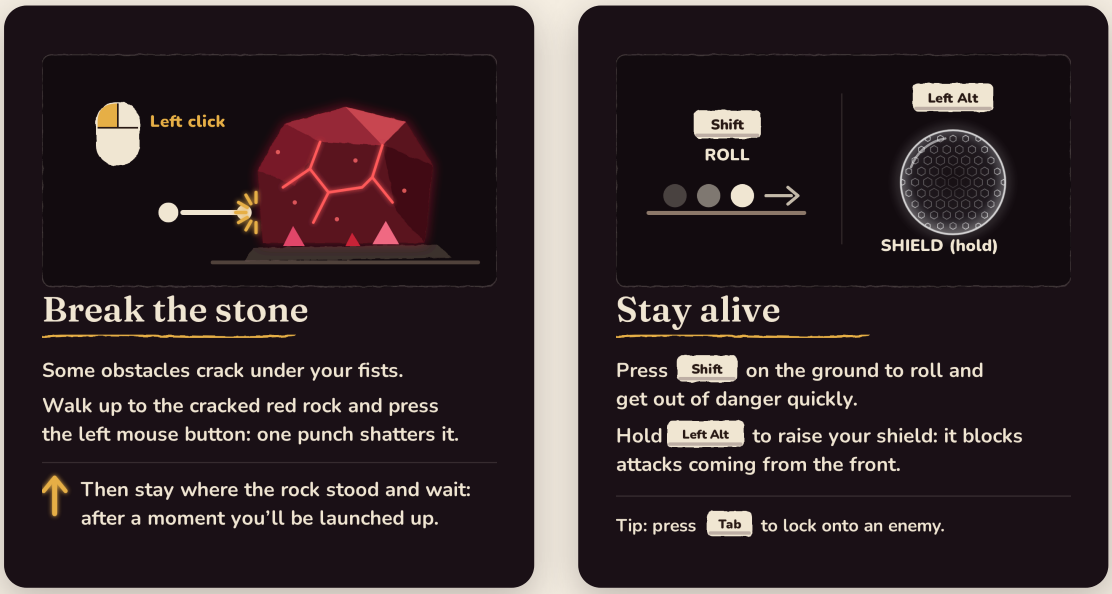


Sezione 7: il corridoio, con le armi che aspettano in fondo.

Insegnare senza interrompere

Raggiungendo alcuni punti si sblocca una breve scheda illustrata. Tutte le schede già viste si possono riaprire in qualsiasi momento con T. In arena i nemici aspettano finché non chiudi le schede del combattimento: lo scontro parte quando sei pronto.

Se muori, riparti dall'ultima sezione raggiunta, con tutto ciò che avevi risolto ancora risolto. La cinematica si può saltare e, dopo una morte, non viene più mostrata.

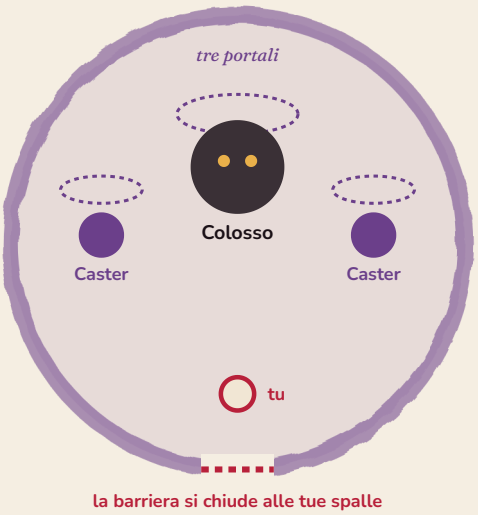


Due delle schede degli hint, come appaiono nel gioco.

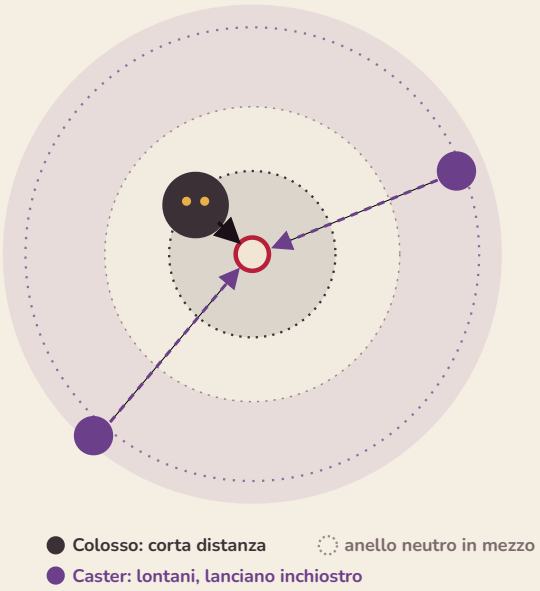
L'arena

Il mio riferimento era il combattimento di *Kingdom Hearts 2*: in ogni istante si dovrebbe capire cosa sta facendo ogni nemico, senza tempi morti in cui non succede niente di leggibile. Lo scontro si basa su due nemici con ruoli opposti.

All'inizio



Durante lo scontro



Il Colosso

Lento, pesante e impossibile da interrompere. Non puoi scambiare colpi con lui: devi girargli intorno e punirlo dopo i suoi attacchi.

I Caster

Fragili, mantengono le distanze e lanciano inchiostro. Ognuno che sconfiggi lascia una goccia d'inchiostro, e uno nuovo esce da un portale.

Fase 1	*Scudo*	*Fase 2*
Colosso 100% → 50%	**Colosso protetto**	**Colosso 50% → 0%**
I Caster continuano a tornare, l'inchiostro si ricarica.	Solo un colpo caricato con il pennello lo rompe. Costa tre gocce: sigillo pieno vuol dire che sei pronto.	Niente più Caster. L'inchiostro finisce.

A che punto è

Non ho raggiunto del tutto l'obiettivo. Lo scontro funziona e si può completare, ma in alcuni momenti l'intenzione di un nemico non è chiara e a tratti il ritmo cala. È la prima cosa su cui tornerei (capitolo 7).

Dalla matita alla *pittura*

Ogni modello del gioco nasce da un mio disegno. Gli strumenti di IA trasformano il disegno in una base 3D; io scelgo l'input, confronto gli strumenti e rifinisco il risultato a mano.

1	2	3	4	5	6
Schizzo	FLUX.2 [klein]	Tripo AI	Hunyuan3D	PixelPasto 2.0	GIMP
matita o digitale	ComfyUI, in locale	modello 3D	texture di base	Blender, effetto dipinto	correzione colore

L'angelo



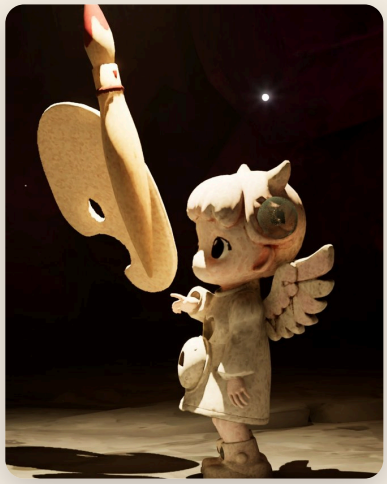
Lo schizzo a matita.

→



FLUX.2 [klein]: un'immagine pulita, pronta per il 3D.

→



Nel gioco.

Un pezzo alla volta

Per ottenere modelli puliti da Tripo AI ho diviso il personaggio in parti (testa, corpo, vestito, ali, mani, scarpe), le ho generate separatamente e poi le ho riunite in Blender.

Ho provato anche **TRELLIS 2** in locale sulla mia RTX 4070, ma i risultati non erano all'altezza di **Tripo AI**, il servizio a pagamento che ho scelto.



La testa, generata da sola.

Gli oggetti di scena, allo stesso modo

Gli oggetti di scena seguono la stessa strada: un veloce schizzo digitale, un'immagine pulita con FLUX.2 [klein], poi Tripo AI.



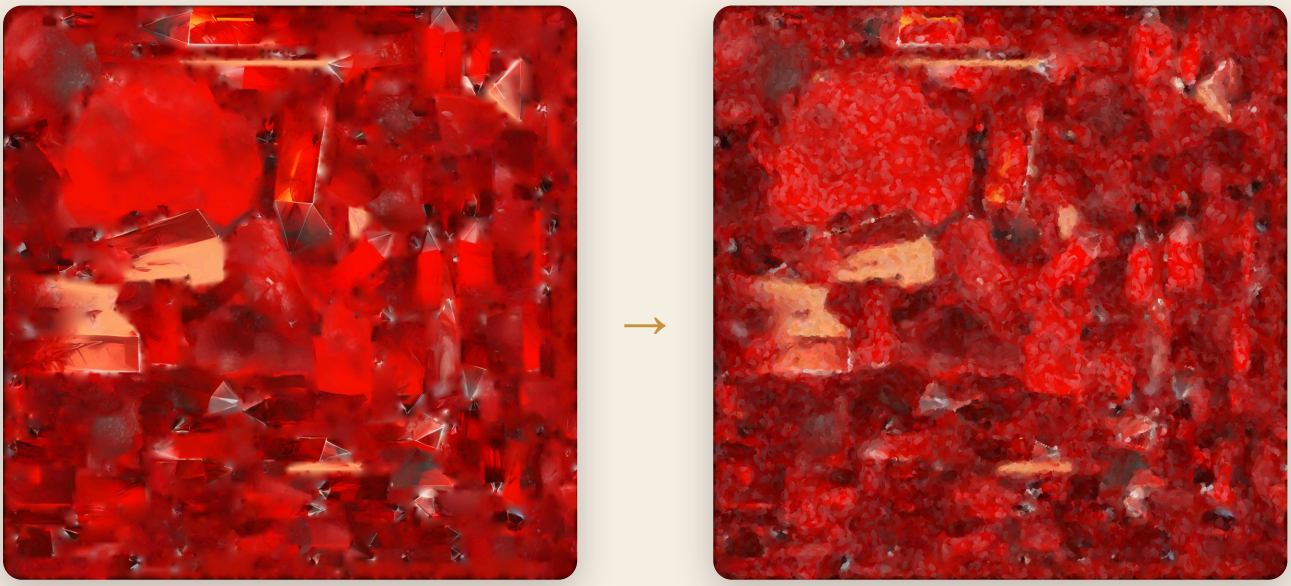
Due oggetti di scena: lo schizzo, poi l'immagine inviata a Tripo AI.

Topologia pulita, possibile da poco

Fino a poco tempo fa le mesh generate dall'IA erano dense e disordinate, difficili da riggare e animare. Le cose sono cambiate solo negli ultimi mesi: prima Hunyuan3D ha introdotto la retopologia automatica, poi Tripo AI ha rilasciato **Smart Mesh**, che genera modelli con topologia pulita. Generando il personaggio a pezzi e riunendoli in Blender, ogni parte esce pronta all'uso.

L'effetto dipinto

Sono le texture a dare al gioco l'aspetto dipinto, e nascono in due passaggi. **Hunyuan3D** (gratuito, di Tencent) crea una texture di base dall'immagine e dal modello 3D. **PixelPasto 2.0**, un setup di Blender a pagamento (un file .blend), la ridipinge con pennellate visibili. Quando i colori deviavano, ho corretto tonalità e temperatura in GIMP.



Texture di base da Hunyuan3D.

Dopo PixelPasto 2.0.

Ogni asset del gioco è passato da questo trattamento. È quella pennellata comune a tenere insieme il mondo.

Dare loro *vita*

Gran parte delle animazioni dell'angelo viene da pacchetti gratuiti di Fab e dalla libreria di Unreal. Il mio lavoro è stato farle funzionare su un corpo per cui non erano state pensate.

Rig e pesi

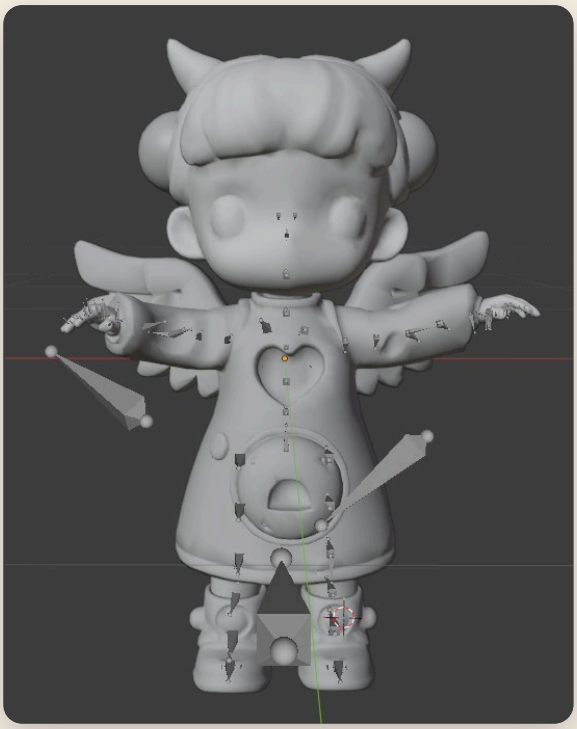
Rig con **AccuRig**, pesi dipinti a mano. Il vestito resta rigido, come pietra scolpita; le compenetrazioni le ho risolte direttamente sul modello.

Retargeting

Le animazioni erano fatte per il manichino di Unreal. Ho impostato il retargeting sulle proporzioni chibi dell'angelo, correggendo la mappatura che lo faceva fluttuare e teletrasportare.

Logica delle animazioni

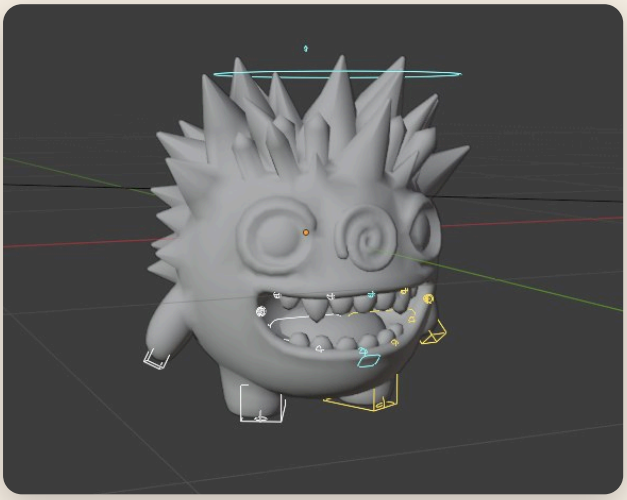
Un solo Animation Blueprint gestisce camminata, salti, doppio salto, posture armata e disarmata e la guardia, con passaggi fluidi tra loro. Gli attacchi sono montage divisi in sezioni, così le combo si possono concatenare.



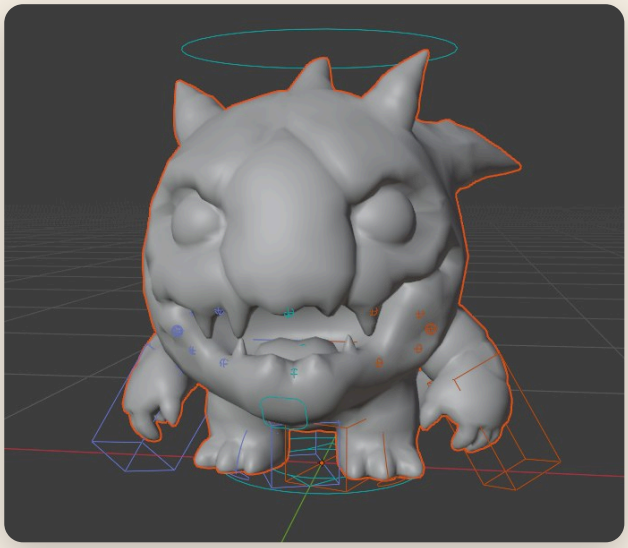
Il rig dell'angelo in Blender.

I nemici

Rig con **Auto-Rig Pro**; ho sistemato i pesi a mano, correggendo le trappole delle mesh generate dall'IA, come i vertici duplicati che si strappano quando il corpo si muove. Le animazioni (5 per il Caster, 9 per il Colosso) le ha scritte **Claude** tramite script Python in Blender, sotto la mia regia: descrivevo ogni movimento, controllavo i render e chiedevo le modifiche.



Il Caster, riggato in Blender.



Il Colosso, riggato in Blender.

Caster 5 clip

Idle	respiro lento, bocca inquieta
Camminata	in loop, con la mandibola
Lancio	caricamento e tiro dall'alto
Colpito	rinculo, un piede che arretra
Morte	cade all'indietro, scalciando

Colosso 9 clip

Idle	respiro pesante
Camminata	in loop
Colpito	barcollamento lento e faticoso
Manata	torsione del busto e colpo
Inizio carica	si accuccia e scatta in avanti
Carica	in loop, braccia che pompano
Stordito	inciampa e barcolla dopo la carica
Ruggito	tre rimbalzi, poi il ruggito
Morte	si ribalta

> **!** **La parte più fragile**
>
> L'animazione è probabilmente l'area più debole del progetto. Un personaggio così lontano dall'umanoide standard, con una testa enorme, arti corti e un vestito rigido, avrebbe meritato il lavoro di un animatore professionista.

Due sistemi, *da vicino*

Tutto il gioco è costruito in Blueprint, e il materiale è molto più di quanto stia qui. Ho scelto due sistemi da mostrare nel dettaglio; per tutto il resto, scrivetemi pure.

1 · Interagire con gli oggetti

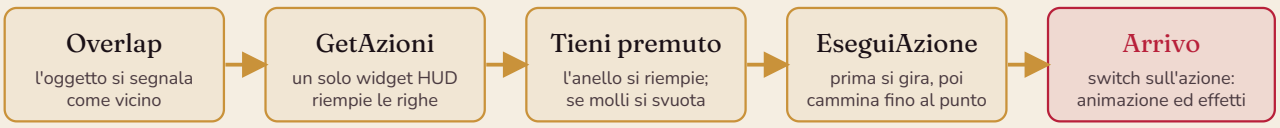
Ogni oggetto interattivo implementa un'interfaccia, **BPI_Interagibile**, con tre funzioni:

- `GetAzioni` restituisce l'elenco delle azioni: tasto, etichetta e nome dell'azione.
- `EseguiAzione` avvia un'azione dal suo nome.
- `GetInteractionLocation` restituisce il punto in cui il personaggio deve arrivare.

L'oggetto decide *cosa* si può fare: il cristallo offre *Ruota* e *Ascolta*, e widget, tasti e anello di caricamento si riempiono da quell'elenco. Il personaggio decide *come* si vede: cammina fino all'oggetto e all'arrivo uno switch sul nome dell'azione avvia animazione ed effetti. Un oggetto nuovo riusa tutto questo; un nuovo tipo di azione aggiunge un caso allo switch.



L'elenco delle azioni arriva dal cristallo.



Un widget, non uno per oggetto

Un widget su ogni cristallo finiva dietro la mesh e fluttuava con lei. Un solo widget sull'HUD, proiettato sul punto dell'oggetto, ha risolto entrambe le cose.

Prima girarsi, poi camminare

Girarsi e camminare insieme faceva girare il personaggio attorno al cristallo all'infinito quando partiva di spalle. Due fasi separate l'hanno risolto.

2 · Targeting e allungo

Nei primi playtest i Caster erano quasi impossibili da colpire: scappano per design, ogni colpo li respingeva e il pennello andava a vuoto. Due elementi hanno cambiato completamente lo scontro: a ogni colpo della combo il personaggio sceglie un bersaglio, e l'animazione si allunga per raggiungerlo.

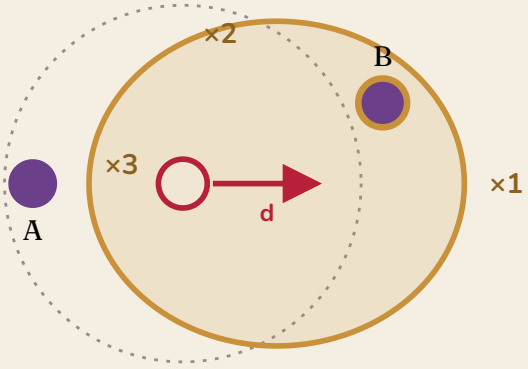
Scegliere il bersaglio

A ogni colpo, `TrovaBersaglio` assegna un punteggio a ogni nemico vivo nelle vicinanze e sceglie il più basso. Se il giocatore ha agganciato un bersaglio con Tab, vince quello senza alcuna ricerca.

$$\mathbf{v}_e = \left(p_e - p_{\text{self}}\right)_{xy}, \qquad \hat{\mathbf{d}} = \begin{cases} \widehat{\text{input}} & \|\text{input}\| > 0.1 \\ \widehat{\text{forward}} & \text{altrimenti} \end{cases}$$

$$s(e) = \|\mathbf{v}_e\| \left(2 - \hat{\mathbf{v}}_e \cdot \hat{\mathbf{d}}\right) \qquad e^* = \underset{e \in E}{\arg\min}\ s(e)$$

- **p** sono posizioni: del giocatore (*self*) e del nemico *e*. $\mathbf{v_e}$ è il vettore dal giocatore al nemico sul piano del terreno (altezza ignorata); la sua lunghezza è la distanza.
- **d** è dove il giocatore vuole andare: l'input di movimento se sta spingendo, altrimenti la direzione in cui guarda il personaggio.
- Il **prodotto scalare** dei due vettori unitari è il coseno dell'angolo tra loro: 1 davanti, 0 di lato, −1 dietro.
- **2 − cos** lo trasforma in un moltiplicatore da 1 (davanti) a 2 (lato) a 3 (dietro): un nemico alle spalle conta come tre volte più lontano.



Stesso punteggio su tutta la curva

Un'ellisse con il giocatore in un fuoco, allungata lungo d: davanti arriva tre volte più lontano che dietro.

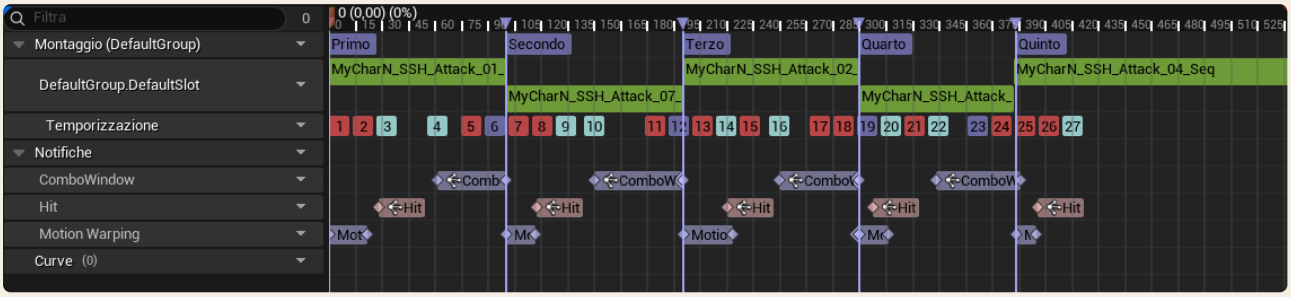
A è più vicino, ma dietro: fuori.
B è più lontano, ma davanti: dentro.

Vince B.

In grigio tratteggiato: la semplice distanza, per confronto.

Raggiungerlo: Motion Warping

Il primo tentativo spostava il personaggio con una timeline: sembrava un teletrasporto e combatteva contro il root motion dell'animazione. La soluzione è stata lasciare il lavoro all'animazione. Ogni attacco fa già un passo in avanti; con il **Motion Warping** quel passo si allunga fino a raggiungere il nemico.



Il montage della combo a cinque colpi. Nella riga Motion Warping, ogni sezione ha una finestra di warping che termina poco prima del suo notify Hit.



Le distanze si misurano fino al bordo della capsula del nemico, così la stessa regola funziona per il piccolo Caster e per l'enorme Colosso. Solo a terra.



La combo ha scelto il Caster: il passo si allunga e il pennello va a segno.

Da qui lo scontro è cambiato: la combo scorre da un nemico all'altro invece di inseguirli.

Dipingere con la *luce*

L'effetto di cui vado più fiero è l'inchiostro incandescente che il pennello lascia nell'aria durante l'attacco speciale: un ribbon Niagara e un materiale su misura.



L'attacco speciale: la pennellata resta dipinta nell'aria, poi si stacca.

Il ribbon

- **World space:** la scia resta dove è passato il pennello, così sembra dipinta nell'aria invece che trascinata dietro.
- **Texture a scala costante:** le UV si ripetono in base alla lunghezza, ogni 100 unità, così un colpo veloce non stira mai la pennellata.
- **La vita dell'inchiostro:** ogni punto vive 2 secondi. Nasce scuro e trasparente, diventa più luminoso e pieno verso la coda (fino a quattro volte, in HDR, e accende il bloom), poi si stacca di colpo invece di sfumare.



L'emitter Niagara.

Il materiale

L'aspetto dipinto nasce da tre texture in scala di grigi: una **pennellata**, le **setole** e un **rumore**. Il rumore deforma i bordi, che si strappano come vero inchiostro, e ci scava buchi irregolari; una soglia trasforma poi il risultato morbido in bordi netti, come un pennello asciutto.

$$\mathrm{ink}(u) = \mathrm{clamp}\Big(h \cdot \big(S(u)\,B(u) \;-\; D\,N(u) \;-\; \tau \big),\; 0,\; 1 \Big)$$

$$h = 8, \qquad \tau = 0.35, \qquad D \approx 0.82$$

Per ogni punto *u* della scia: S pennellata, B setole, N rumore (texture, da 0 a 1), D intensità dell'erosione, τ soglia, h durezza del bordo. È una forma semplificata della maschera di opacità del materiale, basata su una tecnica standard, l'**alpha erosion**. Il colore è un cremisi scuro spinto molto sopra 1, così il bloom lo trasforma in un bagliore.

La luce che arrivava in ritardo

Quando l'arena si accende, i cristalli brillano di viola e dovrebbero colorare le rocce intorno. Con Lumen quella luce riflessa arrivava con secondi di ritardo: le superfici emissive illuminano la scena solo attraverso una cache che si aggiorna un po' alla volta. Alzo la frequenza di aggiornamento della cache solo per i due secondi in cui l'arena si accende, poi la riporto al valore normale. Il bagliore ora raggiunge subito le rocce, senza pagarne il costo per il resto del gioco.

Interfaccia

La direzione artistica è mia: un velo d'inchiostro con un bordo di porcellana dipinto, la palette e i due caratteri usati in questo diario. Widget e texture sono stati realizzati con Claude seguendo queste regole. Il sigillo dell'inchiostro riempie un vertice per ogni goccia raccolta: un triangolo di cerchi in linea con l'aspetto mistico e dipinto del gioco.



Uno strumento, non una *scorciatoia*

La mia formazione è nell'intelligenza artificiale, quindi l'ho usata come userei qualsiasi strumento: dove fa risparmiare tempo, controllata a ogni passo, e mai come autrice delle decisioni.

Area	Strumenti	Cosa ha fatto l'IA	Cosa ho fatto io
Grafica	**FLUX.2 [klein], Tripo AI, Hunyuan3D, PixelPasto 2.0**	Immagini pulite, basi 3D, texture di base	Disegni, scelta e confronto degli strumenti, divisione in parti, assemblaggio in Blender, correzione colore
Gameplay	**Claude**	Guida tecnica: progettazione dei sistemi insieme, debug, documentazione	Ogni Blueprint di gioco, costruito e tarato a mano nell'editor
Interfaccia	**Claude Code dentro Unreal**	Widget, materiali e texture della UI, analisi degli asset	Direzione artistica, palette e caratteri, revisione di ogni asset
Animazione	**Claude in Blender**	Le animazioni dei nemici tramite script Python	Descrizione di ogni movimento, revisione dei render, pesi, integrazione in Unreal

Come ci ho lavorato

Gli strumenti di IA funzionano meglio quando restano strumenti sotto il tuo controllo, non qualcosa a cui chiedere di costruire un intero sistema da zero. In questo progetto la direzione artistica, le scelte tecniche e la programmazione sono sempre rimaste a me.

Questo mi ha permesso di andare veloce, da solo, senza altre figure specializzate. Non le sostituisce: l'animazione e le aree che richiedono competenze profonde, come la luce o l'IA dei nemici, hanno ancora bisogno di persone esperte.

Cosa *terrei*, e cosa cambierei

Cosa ha funzionato

- **La pipeline grafica.** Tre mesi, una persona, e un mondo dipinto coerente.
- **Targeting e allungo.** Il momento in cui lo scontro ha cominciato a funzionare.
- **Un prologo completo.** Dall'inizio alla fine, con checkpoint, hint, una cinematica e un finale.

Cosa cambierei

- **Leggibilità dello scontro.** Le intenzioni dei nemici non sono sempre chiare, e a tratti il ritmo cala.
- **Animazione.** La parte più fragile; serve l'occhio di un professionista.
- **Audio.** Ancora non c'è, ed è lo scontro a sentirne di più la mancanza.

Prossimi passi

- **C++.** Portare un sistema, a partire dall'IA dei nemici, da Blueprint a C++.
- **Audio e feedback.** Il suono, e un feedback dei colpi più forte nello scontro.
- **Rifinire l'arena.** Prima la leggibilità, poi il ritmo.

Grazie per la lettura.

Giovanni Mantegna · gmantegna96@gmail.com

Guarda il gameplay